

关于 711_POS_System 的学习文档

1. Vs code 的环境配置问题：通过 B 站，CSDN 社区及询问 AI 等途径，完成环境配置并成功打出 hello world。
2. 因项目时间限制而 C 语言内容较多，学习全部比较困难：询问 AI，汇总需要用到的知识点，通过 B 站的 C 语言教程与借阅《C Primer Plus》学习基本内容，并边做边学。
3. 提示词中含中文导致终端出现中文乱码：借用网络寻求解决方案，利用 `system("chcp 65001")` 把编码切换到 UTF-8。
4. 中英文输入法中的部分符号及空格不一样，有时输完提示词后忘记切换就会报错。
5. 关于 “=” 与 “==” 的区别：=代表赋值，而==才是等于，是判断。
6. 数组长度：每个字符串结尾必须带一个空字符（\0），用来告诉程序字符串到此结束，所以 `char name[10]` 实际上只能存九个有效字符。
7. 结构体是一个盒子，可以装入不同的东西，而 `int, float, char` 等只能装特定类型的东西，在本题中，将条码，价格等统一放入统一集中，容易查找。
8. 在代码中，要用到一个变量时必须提前声明，告诉电脑这个变量的存在。且应该对该变量进行初始值的规定，因为从如果不规定，电脑不知道初始是多少，会在里面随便塞一个数字，同样，在输入完成后同样要告诉电脑现在只有三个货物，不然可能会产生内存垃圾值。
9. 变量命名：要取一个容易看懂的变量名，如果取 `a, b, c` 等，在复杂程序中可能做着做着会不太记得该变量到底是什么。且变量命名中间不能有空格（空格用来分开不同单词），一般用 `_` 代替空格。
10. 函数要先定义，再调用。在定义函数时，需要以返回类型 函数名 （参数列表）{函数体} 的形式定义，`void` 在返回类型中代表这个函数无需返回给电脑一个具体数值，而参数列表中的 `void` 代表这个函数不需要输入参数（此时 `void` 可以省略），如 `show_prices`，本身就不需要传参数。而在调用函数时格式应为函数名（传入的参数），括号内不能填 `void`。
11. C 语言类似于一个总分的结构，需要先告诉电脑这是什么（要先声明，要初始化，这样才能保证电脑不会因为一些量未知而乱赋值，造成错乱），再细说它的内容，最后再用它去实现功能。且 C 语言中数据是动态变化的，程序会根据变化而进行下一步操作，捕捉变化，及时响应。
12. `Inventory[o].name="Cola"` 不可行，因为 `name` 是一个常量指针，代表内存首地址，不能给地址赋值，所以需要 `strcpy` 将内容复制进去。
13. `Inventory[i].name` 的写法含义：`inventory` 是我定义的结构体的数组名，就像一个仓库，用来储存东西，`[i]` 是一个下标，代表仓库里的第 `i` 个商品，`.name` 中的 `.` 是成员访问运算符，意思是打开此盒子，取出 `name` 这个变量。
14. 显示宽度和字节宽度：字节宽度是指数据在内存和硬盘中占据的物理大小，英文，数字及半角符号占 1 个字节，在 UTF-8 中，中文占 3 个字节。而显示宽度则是指的视觉格子数，在视觉上，英文，数字及半角符号占一个格子，中文和全角符号占两个格子。所以如果在编码中同时使用中文和英文且不做其他处理的话，容易产生排版错乱。
15. 加购，减购及打印小票的基本逻辑均是先把函数及其内容写出来，再在主循环中调用函数来实现功能。
16. 加购及减购中逻辑需要清晰，加购要分为购物车中原来有，没有和商品不存在三种情况，所以应该首先遍历所有商品，根据有没有分流，再在有这个商品时遍历购物车中是否有这个商品，若有，则该商品数量加一，若没有，则新加该商品。（但在这里我遇到了一个问题，那就是重复计算出现的问题，如若打出 001 001，我的终端不是如示例一样打出 Cola 3.50x2=7.00，而是会打出 Cola 3.50x1=3.50 Cola 3.50x2=7.00 两行，由于该问题不影响大体功能且时间有限，我没有再做修改）

17. 而在减购中，逻辑类似，在减购完一件后若仍有剩余则只需数量-1即可，而若是减购完一件购物车内已不存在该商品则需要将该商品从购物车中删除，C语言中没有特定的删除函数，所以删除操作应为将该商品后的商品序号都前进一，同时将购物车中的商品数量减一，利用移动覆盖来达到删除的目的，这样以后遍历只会看前 n-1 个（假设原来购物车内有 n 个商品），就不会在遍历那个被删除了的商品。

18. break 与 return 的使用区别：break 只是跳出内层循环，而这一层循环外的下面的代码可以接着运行，而 return 则是直接终止，下面的代码都不再运行。这两者使用错误就有可能造成代码逻辑不对，而在本题中有部分代码由于 if 的原因（如 add to cart 函数定义中的某个），由于 if 内条件不成立时该代码不会运行，所以以我目前的进度，我认为在这里用 break 或 return 可以，不过这只是从效果上来说，，从严谨以及后续操作来讲，还是应该用 return。

19. 大括号位置所引起的逻辑穿透:我的 remove from cart 的函数定义中，曾因为部分代码位置写的不对而引起逻辑穿透（如写错了 printf("商品已删除")的位置,导致商品已删除和 Item not in the cart 同时出现，以及因为一开始我将 cart count=cart count-1 写进了 for 循环里面，导致在我将一个商品数量减为零成功出现商品已删除后，下一次不管我再输入什么-代码（甚至这个条码是 00 或 004 这种不存在于商品列表中的条码），均会出现商品已删除。这让我明白了大括号作用域问题的严重性，尤其是在这种多个循环嵌套式，大括号数量较多，更容易混淆，导致逻辑错误。

20. while 与 for: 这两个均代表了循环与重复执行，核心逻辑都是只要（）内的条件成立，就会反复执行{}内的代码，区别在于写法的不同。for（1 初始化 2 条件判断 3 更新），一般用于事先知道要执行几次的情况，而 while 则把这些东西分开写，while 的（）内只有条件，初始化写在前面，更新写在循环体内，只要条件为真，便一直执行，一般用于不知道要执行多少次时。本题中的 while(1)中的 1 便是条件，这是一个死循环，在 C 语言中 0 表示假，任何非零数字均代表真（包括负数，只要非零），所以 1 这个条件永远为真，程序会一直执行。死循环不一定错，可以配合 break 或 return 在适合时跳出。

21. Prod 是 product 的缩写，是根据 struct product 创建的一个变量，prod 是一个具体的变量，不过它的类型是 product，在定义后 prod 会拥有 struct product 里定义的所有字段。Struct product 是一个模板，而 prod 是按照这个模板填出来的具体的数据。利用 prod 可以直接将商品资料打包，整体利用，不用一个个复制。

22. Float 与 double: 均是浮点数类型，但二者的精度和占用内存不同。Float 是单精度，取值范围大概是 -3.4×10^{38} 到 3.4×10^{38} ，内存占 4 字节。而 double 双精度，取值在 -1.7×10^{308} 到 $+1.7 \times 10^{308}$ 之间，精度更高，占 8 字节。